

Red-Black Tree

Souvik Mondol - 2005069
S.M Kausar Parvej - 2005076
Sagor Chanda - 2005071

Department of CSE
Bangladesh University of Engineering and Technology

February 20, 2024

Objectives

- 1 Introduction
- 2 Define the red-black tree properties
- 3 Rotations
- 4 Insertion
- 5 Deletion
- 6 Application

Figure

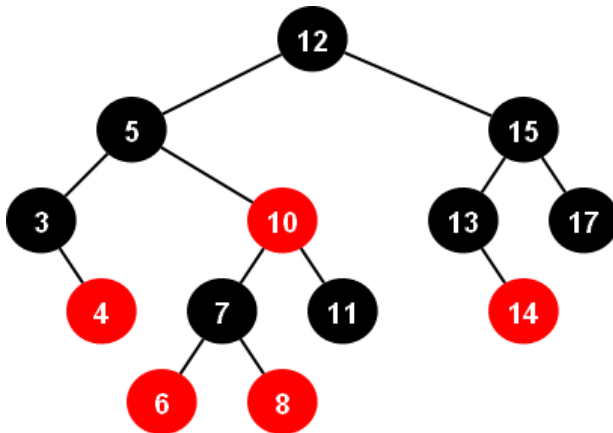


Figure: A Image of **Red-black** Tree

What is a Red-Black tree?

What is **Red-Black** Tree?

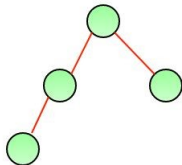
Definition of Red-Black Tree

Red-Black tree is a **Binary Search Tree** in which every node is colored with either red or black.

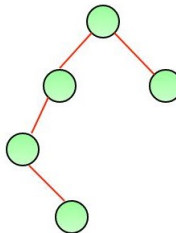
It is a type of self balancing binary search tree. It has a good efficient worst case running time complexity.

Binary Search Trees - Performance

- Insertion and removal from BSTs is $O(\text{height})$
- What is the height of a BST?
 - If the tree is balanced: $O(\log n)$
 - If the tree is very unbalanced: $O(n)$



Balanced BST



Unbalanced BST

Balanced BSTs

- Define a balanced binary tree as one where
 - There is no path from the root to a leaf that is more than twice as long as any other such path
 - The height of such a tree is $O(\log n)$
- Guaranteeing that a BST is balanced requires either
 - A more complex structure (2-3 and 2-3-4 trees) or
 - More complex insertion and deletion algorithms (red black trees)

Red-Black Tree Structures

- A red-black tree is a balanced BST
- Each node has an extra colour field which is
 - **Red- Black**
 - Usually represented as a boolean– **isBlack**
- Nodes have an additional pointer to their parent
- A node's null child pointers are treated as if they were black nodes
 - These null children are imaginary nodes so are not allocated space
 - And are always coloured black

Red-Black tree nodes

- Red-black trees are reference structures
- Nodes contain data, three pointers to nodes, and the node's colour

Red-Black tree properties

- ① Every node is either **red** or **black**
- ② Every leaf is **black**
 - Leaves refers to the imaginary nodes
 - i.e. every null child of a node is considered to be a black leaf
- ③ If a node is **red** both its children must be **black**
- ④ Every path from a node to a leaf contains the same number of black nodes
- ⑤ The root is **black**-for convenience

Red-Black Tree Height

- The black height of a node, $bh(v)$, is the number of black nodes on a path from v to a null black child
 - Without counting v itself
 - Property 4—every path from a node to a leaf contains the same number of black nodes
- The height of a node, $h(v)$, is the number of nodes on the longest path from v to a leaf
 - Without counting v itself
 - Property 3— a red node's children must be black
 - So $h(v) \leq 2(bh(v))$

Balanced Trees

- It can be shown that a tree with the red-black structure is balanced.
- A balanced tree has no path from the root to a leaf that is more than twice as long as any other such path.
- Assume that a tree has n internal nodes:
 - An internal node is a non-leaf node, and the leaf nodes are imaginary nodes, so n is the number of actual nodes.
 - A red-black tree has $\geq 2bh - 1$ internal (real) nodes.
 - Can be proven by induction (e.g., Algorithms, Cormen et al).
 - But consider that a perfect tree has $2^h + 1$ leaves, bh must be less than or equal to h , and that $2^h + 1 = 2^h + 2^h$.

Red-Black Tree Height

- Claim: A red-black tree has height, $h \leq 2 \cdot \log(n + 1)$.
 - ① $n \geq 2bh - 1$ from claim on the previous slide.
 - ② $bh \geq h/2$ (red nodes must have black children).
 - ③ $n \geq 2h/2 - 1$ (replace bh in 1 with h).
 - ④ $\log(n + 1) \geq h/2$ (take logarithm base 2 of both sides of 3, and add 1).
 - ⑤ $2 \cdot \log(n + 1) \geq h$ (multiply both sides of 4 by 2).
 - ⑥ $h \leq 2 \cdot \log(n + 1)$ (reverse 5).
- Note that $2 \cdot \log(n + 1)$ is $O(\log(n))$.
 - If insertion and removal are $O(\text{height})$, they are $O(\log(n))$.

Rotation

Rotations

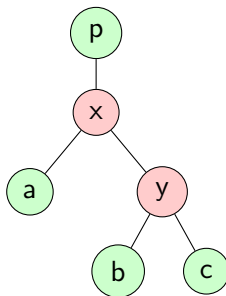
- An item must be inserted into a red-black tree at the correct position.
- The shape of a tree is determined by:
 - The values of the items inserted into the tree.
 - The order in which those values are inserted.
- This suggests that there is more than one tree (shape) that can contain the same values.
- A tree's shape can be altered by rotation while still preserving the binary search tree (BST) property.

Left Rotation

Left Rotation

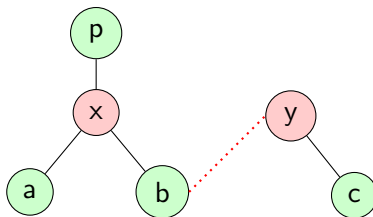
In left-rotation, the arrangement of the nodes on the right is transformed into the arrangements on the left node.

Let the initial tree be:



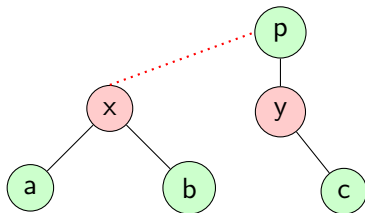
Left Rotation

- If y has a left subtree, assign x as the parent of the left subtree of y .



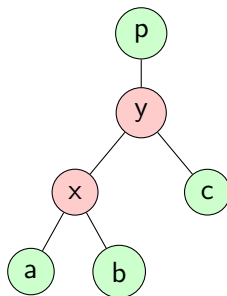
Left Rotation

- If the parent of x is NULL, make y as the root of the tree.
- Else if x is the left child of p , make y as the left child of p .
- Else assign y as the right child of p .



Left Rotation

- Make y as the parent of x.

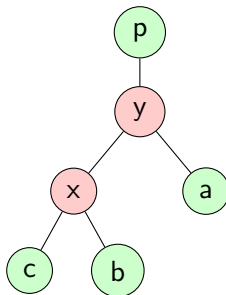


Right Rotation

Right Rotate

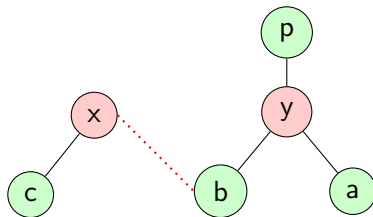
In right-rotation, the arrangement of the nodes on the left is transformed into the arrangements on the right node.

Let the initial tree be:



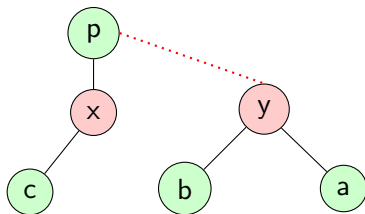
Right Rotate

- If x has a right subtree, assign y as the parent of the right subtree of x .



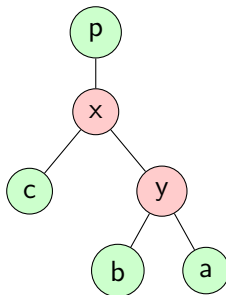
Right Rotate

- If the parent of y is NULL, make x as the root of the tree.
- Else if y is the right child of its parent p, make x as the right child of p.
- Else assign x as the left child of p.



Right Rotate

- Make x as the parent of y.



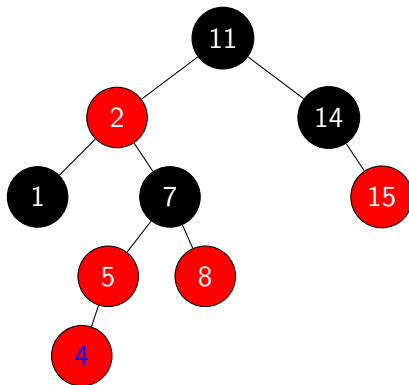
Insertion

Red-Black Tree Insertion Steps

- **Insert as for a BST and make the new node red**
 - The only property that can be violated is that both a red node's children are black (its parent may be red)
- **If this is the case, try to fix it by coloring the new node red and making its parent and uncle black**
 - Which only works if both were red
 - As otherwise the equal black height property will be violated
- **If changing the colors doesn't work, the tree must be rotated**
 - Which also entails changing some colors

Red-Black Tree Insertion Simulation

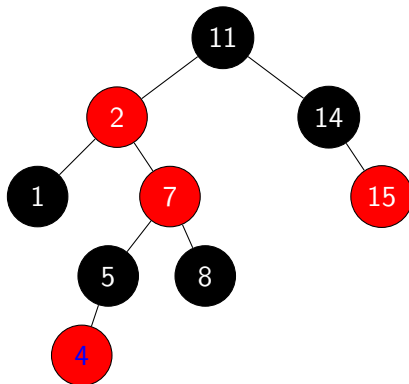
Step 1: Insert as for a BST and make the new node red
(Inserting 4)



Violation Case 1: z and $p[z]$ are both red; z 's uncle y is red

Red-Black Tree Insertion Simulation

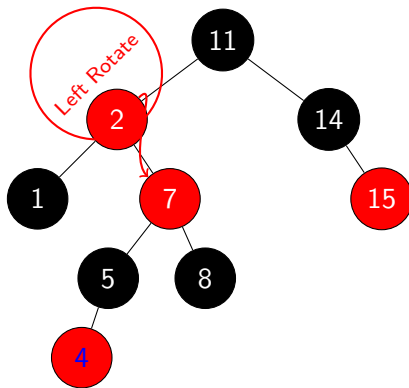
**Step 2: 4, it's parent 5 and uncle 8 all are red
(So making 5, 7 and 8 inverting their color)**



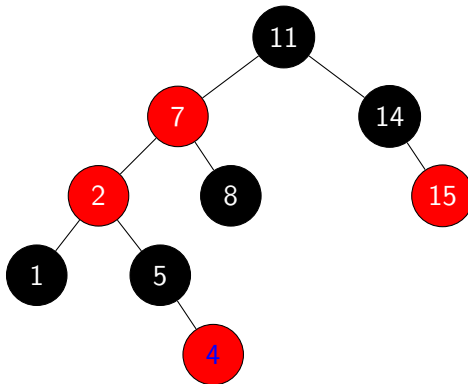
Violation Case 2: z's uncle is black; z is a right child

Red-Black Tree Insertion Simulation

Step 3: Now 7 and it's parent 2 are both red but 7's uncle 14 is black (So Left rotating 2)



Red-Black Tree Insertion Simulation

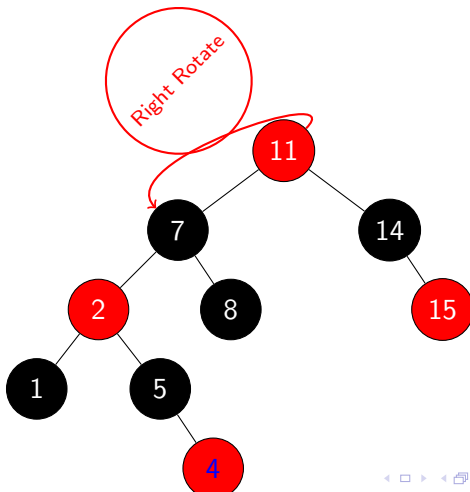


After Left Rotated at node 2

Violation Case 3: z's uncle is black; z is a left child

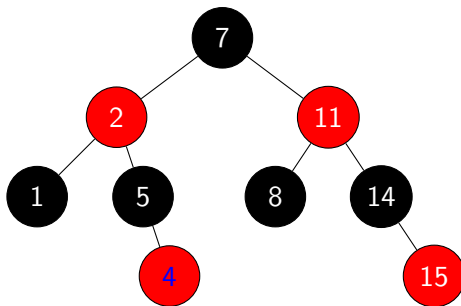
Red-Black Tree Insertion Simulation

**Step 4: Now 2 and it's parent 7 are both red but 2's uncle 11 is black and 2 is a left child
(So inverting the color of 7 & 11 and right rotating 11)**



Red-Black Tree Insertion Simulation

And here the balanced Tree!!



After Right Rotated at node 11

Deletion

Red-Black Trees: Deletion

- Deletion, like insertion, should preserve all the Red-Black properties.
- Steps of deletion:
 - Do regular BST deletion.
 - Fix any violations of Red-Black properties that may result.

Red-Black Trees: Deletion

Deletion is a fairly complex process. Before going through the steps, one must know about **double black**.

Red-Black Trees: Deletion

Deletion is a fairly complex process. Before going through the steps, one must know about **double black**.

Double Black

When a black node is deleted and replaced by a black child, the child is marked as **double black**. The main task now becomes to convert this **double black** to single black.

Steps of Deletion

Step 1

Regular BST deletion

Suppose deleting a node "x".

3 Cases:

- ① x has no children:
 - Remove x
- ② x has one child:
 - Splice out x
- ③ x has two children:
 - Find its inorder successor y
 - Replace x with y
 - Delete y

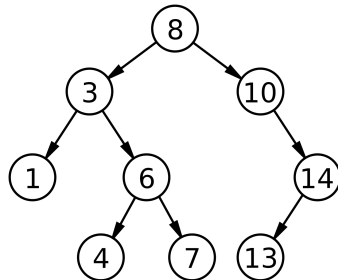


Figure: Suppose delete 3, 1 or 14

Steps of Deletion

Step 2

Fixup violations of Red-Black properties

There are several cases that may occur during fixup operations.

Steps of Deletion

Case 1

If the node is **Red** which is to be deleted, we simply delete it.

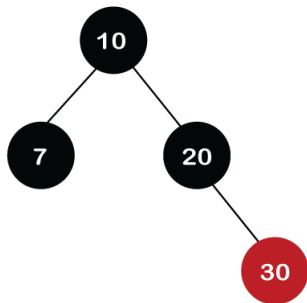


Figure: Before deleting 30

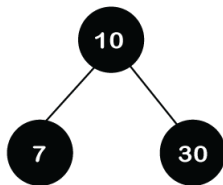


Figure: After deleting 30

Steps of Deletion

Case 2

If the node is a double black, just remove the double black (double black occurs when the node to be deleted is a black colored leaf node, as it adds up the NULL nodes which are considered black colored nodes too)

Case 3

If the double black's sibling node is also a black node and its child nodes are also black in color, follow the steps below:

- Remove double black
- Recolor its parent to black (if the parent is a red node, it becomes black; if the parent is already a black node, it becomes double black)
- Recolor the parent's sibling with red
- If double black node still exists, we apply other cases.

Case 4

If the double black node's sibling is red, we perform the following steps:

- Swap the colors of the parent node and the parent's sibling node.
- Rotate parent node in the double black's direction
- Reapply other cases that are suitable.

Case 5

If the double black's sibling is a black node but the sibling's child node that is closest to the double black is red, follows the steps below:

- Swap the colors of double black's sibling and the sibling's child in question
- Rotate the sibling node in the opposite direction of double black (i.e. if the double black is a right child apply left rotations and vice versa)
- Apply case 6.

Case 6

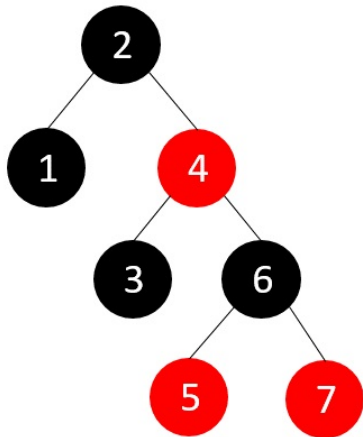
If the double black's sibling is a black node but the sibling's child node that is farther to the double black is red, follows the steps below:

- Swap the colors of double black's parent and sibling nodes
- Rotate the parent in double black's direction (i.e. if the double black is a right child apply right rotations and vice versa)
- Remove double black
- Change the color of red child node to black.

Deletion Example

Delete few elements

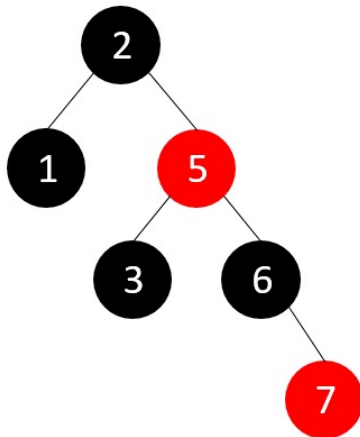
Delete 4



Deletion Example

Delete few elements

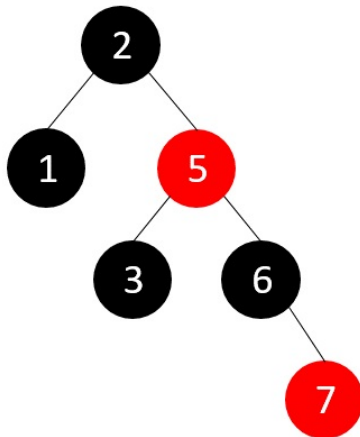
Delete 4



Deletion Example

Delete few elements

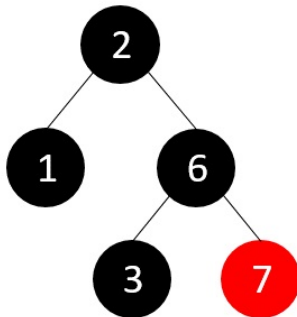
Delete 5



Deletion Example

Delete few elements

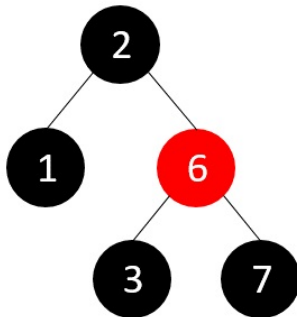
Delete 5



Deletion Example

Delete few elements

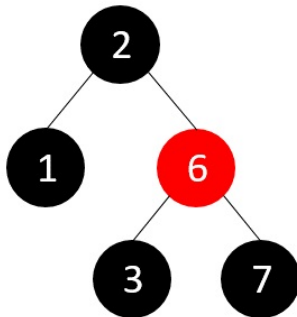
Delete 5



Deletion Example

Delete few elements

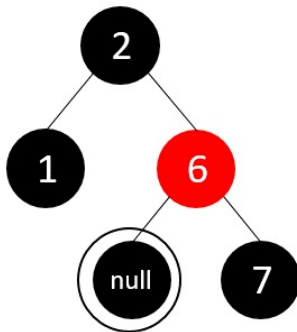
Delete 3



Deletion Example

Delete few elements

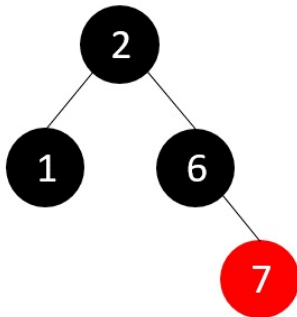
Delete 3



Deletion Example

Delete few elements

Delete 3

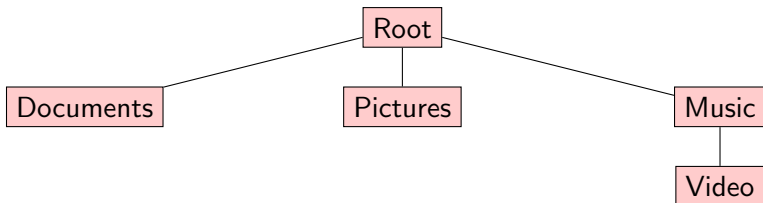


Real-Life Use Cases

Red-Black Trees: Real-Life Use Cases

File Systems

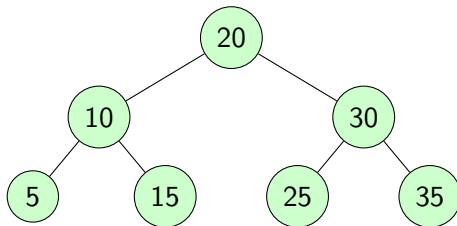
Red-Black Trees are crucial for organizing files efficiently in file systems.



Balanced Trees: Real-Life Use Cases

Database Indexing

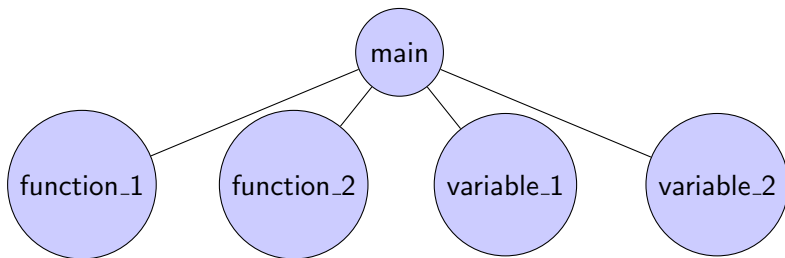
Balanced trees enable quick retrieval of records in databases based on keys.



Balanced Trees: Real-Life Use Cases

Symbol Tables in Compilers

Balanced trees are used to implement symbol tables for quick lookup of identifiers in compilers.

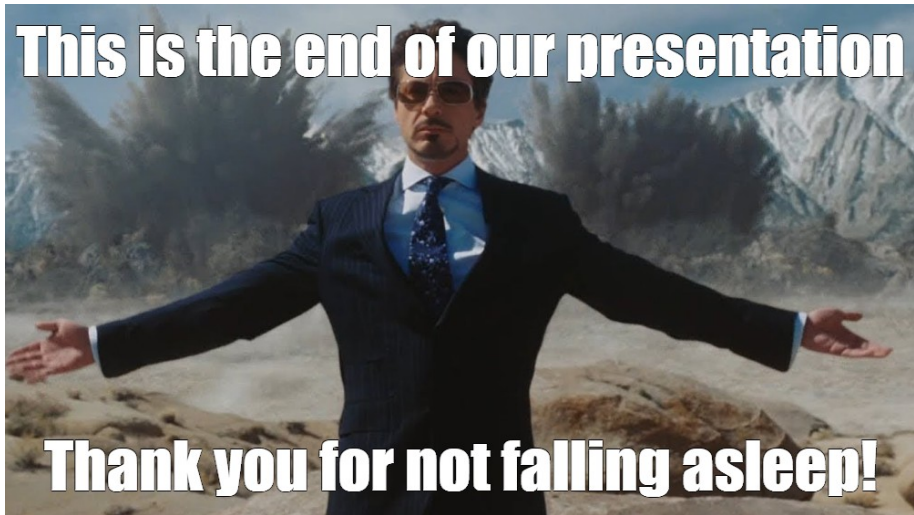


Summary and Conclusion

- Red-Black Trees are an important data structure for maintaining balanced binary search trees.
- They ensure efficient operations by adhering to specific properties.
- The insertion process involves careful balancing to maintain these properties.
- Real-world applications demonstrate the versatility and usefulness of Red-Black Trees.

Questions?

This is the end of our presentation



Thank you for not falling asleep!